Numerical Method - Homework 2

B13902022 Yu-Chi,Lai

1

1.1 (a)

In section 3.4, I found: "Trivially, you can also apply a positive operator to a numeric expression: `int x = +42;` Numeric values are assumed to be positive unless explicitly made negative, so this operator has no effect on program operation"

Hence, the expression 0.0 conveys +0.0 by default. The code and its output is my experiment.

```c
#include <stdio.h>
#include <stdint.h>
#include <string.h>

void print_bits(double d, const char* label) {
    uint64_t bits;
    memcpy(&bits, &d, sizeof(bits));
    printf("%s: %f | Raw Hex: 0x%016lx\n", label, d, bits);
}

int main() {
    double pos_zero = 0.0, neg_zero = -0.0;

    print_bits(pos_zero, "Literal 0.0   ");
    print_bits(neg_zero, "Literal -0.0  ");

    if (pos_zero == neg_zero) {
        printf("\nResult: The == operator says they are equal.\n");
    }

    printf("1 /  0.0 = %f\n", 1.0 / pos_zero);
    printf("1 / -0.0 = %f\n", 1.0 / neg_zero);

    return 0;
}
```

```
Literal 0.0   : 0.000000 | Raw Hex: 0x0000000000000000
Literal -0.0  : -0.000000 | Raw Hex: 0x8000000000000000

Result: The == operator says they are equal.
1 /  0.0 = inf
1 / -0.0 = -inf
```

Figure 1: The output

1.2 (b)

In section 7.12.3.6, there's an `signbit()` macro in C99 standard (And you should also `#include <math.h>`). The `signbit` macro returns a nonzero value if and only if the sign of its argument value is negative.

1.3 (c)

We can declare/specify them using unary operators, positive/negative number division by infinity, zero times positive/negative number or the `copysign()` method in `<math.h>`. The program below is my experiment:

```c
#include <stdio.h>
#include <math.h>

int main() {
    double values[] = {-0.0, 0.0, -1.0 / INFINITY, 1.0 / INFINITY, -1.0 * 0.0, 1.0 * 0.0,
    ↪  copysign(0.0, 1.0), copysign(0.0, -1.0)};

    const char *names[] = {"Literal -0.0", "Literal 0.0", "-1.0 / INFINITY", "1.0 /
    ↪  INFINITY", "-1.0 * 0.0", "1.0 * 0.0", "copysign(0, +)", "copysign(0, -)"};

    int n = sizeof(values) / sizeof(values[0]);

    printf("Method             | Value | signbit()\n");
    printf("-------------------|-------|----------\n");

    for (int i = 0; i < n; i++) {
        printf("%-18s | %4.1f  | %d\n", names[i], values[i], signbit(values[i]));
    }

    printf("\n--- Validation through Division ---\n");
    printf("1.0 / +0.0 = %f\n", 1.0 / values[1]);
    printf("1.0 / -0.0 = %f\n", 1.0 / values[0]);

    return 0;
}
```

```
Method             | Value | signbit()
-------------------|-------|----------
Literal -0.0       | -0.0  | 1
Literal 0.0        |  0.0  | 0
-1.0 / INFINITY    | -0.0  | 1
1.0 / INFINITY     |  0.0  | 0
-1.0 * 0.0         | -0.0  | 1
1.0 * 0.0          |  0.0  | 0
copysign(0, +)     |  0.0  | 0
copysign(0, -)     | -0.0  | 1

--- Validation through Division ---
1.0 / +0.0 = inf
1.0 / -0.0 = -inf
```

Figure 2: The output of the program above.

1.4 (d)

We should adopt the first implementation $((x > y)?x : y)$ because if we set $b = -0.0$, the returned value of the first implementation is +0.0=0.0 (Hence, $c = -\infty < 0$) while the second implementation gives -0.0 (Hence, $c = +\infty > 0$). We can use the program below to prove our claim.

```c
#include <stdio.h>
#include <float.h>

double max1(double x, double y) {
    return (x > y) ? x : y;
}

double max2(double x, double y) {
    return (x < y) ? y : x;
}

int main() {
    double a = -5, b = -0.0;
    double c1 = a / max1(b, 0.0);
    double c2 = a / max2(b, 0.0);
    printf("%f %f\n", c1, c2);
}
```

```
-inf inf
```

Figure 3: The output.